BELLMAN–FORD ALGORITHM

Design and Analysis of Algorithms (DAA) — C Implementation

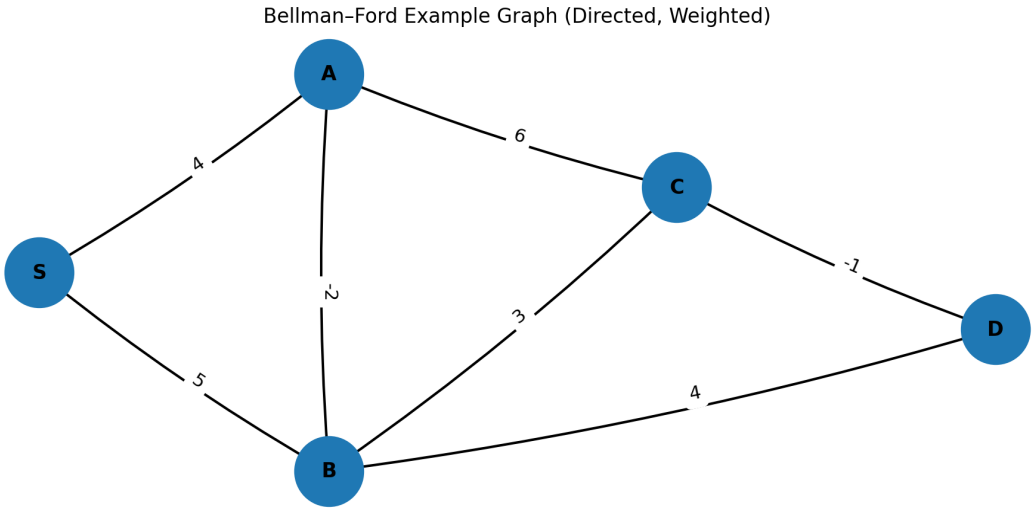
1. Aim

To find shortest distances from a source vertex in a weighted directed graph and detect reachable negative-weight cycles.

2. Algorithm

1. Set source distance to 0 and all other distances to infinity.
2. Relax all edges V−1 times.
3. Stop early if no distance changes.
4. Scan edges once more to detect a negative-weight cycle.
5. Print shortest distances if no such cycle exists.

3. Example Graph



Source: S. Edge labels show weights; negative weights are allowed.

4. C Program (Larger, More Readable Font)

The code is intentionally placed on separate pages in a larger font for easier reading and printing.

```
#include <stdio.h>
#include <limits.h>

#define V 5
#define E 7

struct Edge {
    int src, dest, weight;
};

void bellmanFord(struct Edge edges[], int source) {
    int dist[V];

    for (int i = 0; i < V; i++)
        dist[i] = INT_MAX;
    dist[source] = 0;

    /* Relax all edges V-1 times */
    for (int i = 1; i <= V - 1; i++) {
        int changed = 0;

        for (int j = 0; j < E; j++) {
            int u = edges[j].src;
            int v = edges[j].dest;
            int w = edges[j].weight;

            if (dist[u] != INT_MAX &&
                dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                changed = 1;
```

4. C Program (continued)

```
            }
        }

        if (!changed)
            break;
    }

    /* Check for a negative-weight cycle */
    for (int j = 0; j < E; j++) {
        int u = edges[j].src;
        int v = edges[j].dest;
        int w = edges[j].weight;

        if (dist[u] != INT_MAX &&
            dist[u] + w < dist[v]) {
            printf("Negative-weight cycle detected.\\n");
            return;
        }
    }

    printf("Vertex\\tShortest distance from S\\n");
    for (int i = 0; i < V; i++) {
        if (dist[i] == INT_MAX)
            printf("%c\\tUnreachable\\n", 'S' + i);
        else
            printf("%c\\t%d\\n", 'S' + i, dist[i]);
    }
}

int main(void) {
    /* Vertex IDs: S=0, A=1, B=2, C=3, D=4 */
    struct Edge edges[E] = {
        {0, 1,  4},  /* S -> A */
        {0, 2,  5},  /* S -> B */
        {1, 2, -2},  /* A -> B */
        {1, 3,  6},  /* A -> C */
        {2, 3,  3},  /* B -> C */
        {2, 4,  4},  /* B -> D */
        {3, 4, -1}   /* C -> D */
    };

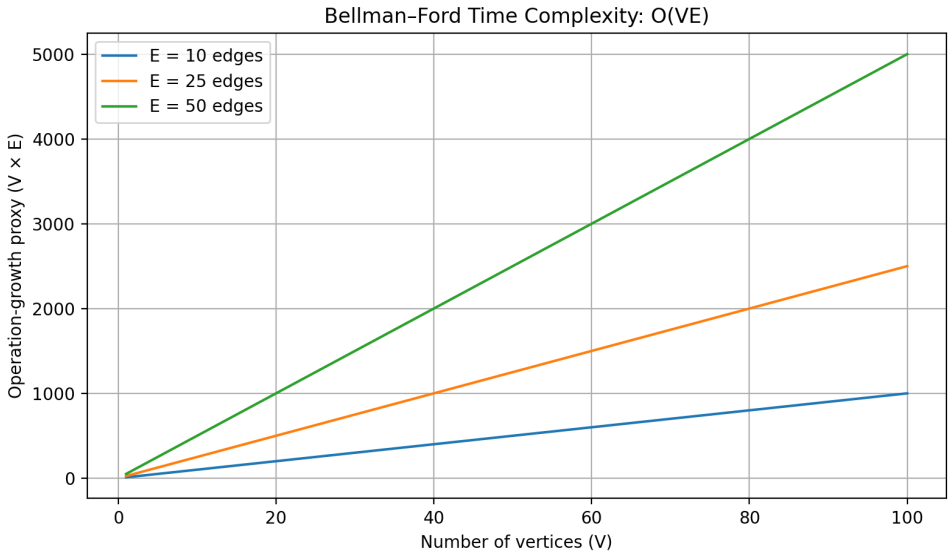
    bellmanFord(edges, 0);  /* Source is S */
    return 0;
}
```

5. Sample Output

```
Vertex   Shortest distance from S
S        0
A        4
B        2
C        5
D        4
```

6. Time and Space Complexity

Measure	Complexity
Worst-case time	O(VE)
Best case with early stopping	O(V + E)
Space (edge list + distances)	O(V + E)



7. Result and Conclusion

The program calculates shortest distances S=0, A=4, B=2, C=5, D=4. Bellman–Ford supports negative edge weights and detects reachable negative-weight cycles. Worst-case time complexity is O(VE).